

Solar plant operations

AI chat assistant — interview task

LLM orchestration

Tool-use pipeline

Multi-table CSV

Estimated time: 2–3 hours

CONTEXT

You are given a dataset of solar-plant operations across three plants in India, split across seven CSV files. Your task is to build a conversational AI assistant that can answer natural-language questions about this data — by designing and implementing an orchestration pipeline on top of an LLM.

The chat interface itself does not need to be polished. A simple terminal REPL or minimal web form is fine. The evaluation is entirely on the quality of the pipeline behind it.

THE DATA

Seven CSV files are enclosed with this task. Each file is a logical endpoint the pipeline can query. The schema forms a two-level hierarchy: plants at the root, inverters as children, and the remaining five tables hanging off one or both.

File	PK	Notes
plants.csv	plant_id	Root table. 3 plants, status field, capacity, tariff.
inverters.csv	inverter_id	Child of plants. 30 inverters, online/offline/fault status.
generation_readings.csv	reading_id	Child of inverters. ~20k rows, hourly, 30 days.
weather_readings.csv	reading_id	Child of plants. ~2.2k rows, ambient + irradiation.
alerts.csv	alert_id	Child of plants/inverters. 29 rows, open/resolved, severity.
maintenance.csv	ticket_id	Child of plants/inverters. 19 rows, scheduled/in-progress/done.
anomalies.csv	anomaly_id	Child of inverters. 55 rows, typed causes, open/resolved.

Foreign keys: plant_id links plants → inverters/alerts/weather_readings/maintenance. inverter_id links inverters → generation_readings/anomalies.

WHAT THE ASSISTANT MUST HANDLE

Type A — current state

status fields

Questions about the present snapshot: which plants or inverters are in a particular status, what alerts are open, what maintenance is in-progress.

e.g. Which plants are currently offline?

e.g. How many inverters are in fault right now?

e.g. What open critical alerts exist?

Type B — statistics & trends

time-series

Aggregations and comparisons over generation and weather readings. Requires multi-row reasoning — sums, averages, correlation, ranking.

e.g. What is the average daily yield per plant over the last week?

e.g. Which inverter has the highest performance ratio?

e.g. What is the mean time to resolve an alert?

Type C — anomaly lookup

inspection log

Queries over the anomaly inspection log, filtering by type, cause, status, or inverter. May require joining with inverters or plants for context.

e.g. Which inverters have open hotspot anomalies?

e.g. What anomalies are caused by soiling?

e.g. Summarise all unresolved anomalies for Rajasthan Solar Park.

PIPELINE REQUIREMENTS

These are the things being evaluated. An LLM calling the data directly is not sufficient — the structure around it is what matters.

1. Intent classification

Before querying any data, the pipeline must classify the incoming question into one of the three types (or a multi-type combination). The classification must be explicit and inspectable — not inferred silently inside a single prompt.

■ **Deliverable:** show the classified intent in the output or logs

2. Tool / function design

Each logical data source must be exposed as a callable tool (function, endpoint stub, or similar). The orchestrator decides which tools to call based on the classified intent — it must not load all seven tables on every question.

■ **Deliverable:** tools must be independently callable and testable

3. Multi-step planning

Some questions require chaining: e.g. 'summarise anomalies for Rajasthan' needs to resolve the plant ID from plants, then filter anomalies. The pipeline must support at least one such multi-step chain.

■ **Deliverable:** demonstrate at least one 2-step chain in your test questions

4. Aggregation in code

The LLM must not be asked to count rows or sum columns from raw CSV text — that is slow, expensive, and unreliable. Aggregations must happen in code (Python, SQL, pandas, etc.) before the result is passed to the LLM for interpretation.

■ **Deliverable:** no raw CSV rows in the final LLM prompt

5. Graceful degradation

If a question is ambiguous or cannot be answered from the available data, the assistant must say so clearly — not hallucinate a number. Include at least one out-of-scope test question.

WHAT YOU DO NOT NEED TO BUILD

- A production-grade UI (a terminal REPL is fine)
- Streaming responses
- Authentication or multi-user support
- A real-time data feed (the CSVs are the full dataset)
- Training or fine-tuning any model

SUGGESTED TEST QUESTIONS FOR THE DEMO

1	Which plant is offline and what is the associated open alert?
2	What was the average daily yield of Rajasthan Solar Park last week?
3	Which inverters have open hotspot anomalies caused by soiling?
4	What is the mean time to resolve a critical alert?
5	What's the weather like at the Gujarat plant today?
6	How much revenue did we lose from Tamil Nadu's downtime this month?

Question 6 is intentionally unanswerable from the data. Watch how the assistant handles it — a clear refusal is correct; a hallucinated number is not.

EVALUATION CRITERIA

Criterion	What we look for
Pipeline structure	Is intent classification separate from data fetching and from LLM synthesis? Are stages composable?

Tool design	Are tools narrowly scoped and testable in isolation? Do they return structured data, not prose?
Aggregation correctness	Are numbers computed in code, not estimated by the LLM?
Multi-step reasoning	Can the orchestrator chain tool calls where the output of one informs the next?
Failure handling	Does the assistant degrade gracefully on ambiguous or unanswerable questions?

Please bring your code and be prepared to walk through the pipeline design decisions during the debrief.